# Avaliação Continuada 04 – Questão 3

Impressão nítida

**Ideia.** No algoritmo recursivo, a única informação que muda entre as chamadas é $i$, o índice da primeira palavra do subproblema ($M$ e $W$ são fixos). Como o mesmo $i$ é resolvido muitas vezes por caminhos diferentes de quebras, memorizamos a resposta de cada $i$ em vetores:

- $custo[i]$: custo mínimo para imprimir as palavras $i, \ldots, n$;
- $prox[i]$: índice $j$ da próxima quebra de linha em uma solução ótima a partir de $i$ ($prox[i] = n+1$ indica que não há quebra).

A recorrência, tirada diretamente das linhas 01–11 do algoritmo recursivo, é

$$custo[i] = \begin{cases} 0, & \text{se } M[i,n] \le W, \\ \min\limits_{\substack{i<j\le n \\ M[i,j-1]\le W}} \Big\{(W - M[i,j-1])^2 + custo[j]\Big\}, & \text{caso contrário.} \end{cases}$$

O conjunto do min nunca é vazio, pois $M[i,i] = L[i] \le W$ (o candidato $j = i+1$ sempre é válido). A resposta do problema é $custo[1]$.

## (a) Número de subsoluções

São armazenadas $n$ subsoluções: uma para cada $i \in \{1, \ldots, n\}$, isto é, $custo[1..n]$ (e o respectivo $prox[1..n]$). *Justificativa:* todo subproblema gerado pela recursão é "imprimir as palavras $i, \ldots, n$", identificado unicamente pelo parâmetro $i$, que só assume valores entre 1 e $n$ (a chamada inicial usa $i = 1$ e as recursivas usam $j \in \{i+1, \ldots, n\}$). Portanto há apenas $n$ subproblemas distintos, embora a recursão sem memorização os resolva um número exponencial de vezes.

## (b) Caso base

O caso base corresponde às linhas 01–02: para todo $i$ tal que $M[i,n] \le W$, as palavras $i, \ldots, n$ cabem em uma única linha, que é a última e tem custo zero. Assim,

$$custo[i] = 0 \quad \text{e} \quad prox[i] = n+1 \qquad \text{para todo } i \text{ com } M[i,n] \le W.$$

Como $M[i,n]$ decresce quando $i$ cresce, esses índices formam um sufixo $\{i_0, \ldots, n\}$; em particular, $i = n$ é sempre caso base, pois $M[n,n] = L[n] \le W$.

## (c) Ordem de preenchimento

A tabela é preenchida com $i$ **decrescente**, de $n$ até 1.

*Argumento:* $custo[i]$ depende apenas de $custo[j]$ com $j > i$ (na recorrência, $j$ varia de $i+1$ a $n$). Quando calculamos a posição $i$, todas as posições $i+1, \ldots, n$ já foram preenchidas, seja pelo caso base, seja em iterações anteriores. Logo nenhum elemento ainda não preenchido é acessado. A matriz $M$ é preenchida antes, pelo algoritmo do enunciado.

# (d) Pseudocódigo

**Algoritmo 1** IMPRESSÃO-NÍTIDA-PD$(L, n, W)$

**Entrada:** tamanhos $L[1..n]$, com $L[i] \leq W$, e largura $W$
**Saída:** custo mínimo e conjunto de quebras de uma solução ótima

```
1:  for i ← 1 até n do                                   ▷ preenche M em O(n²)
2:      M[i, i] ← L[i]
3:      for j ← i + 1 até n do
4:          M[i, j] ← M[i, j − 1] + L[j] + 1
5:  for i ← n até 1 passo −1 do                          ▷ ordem decrescente
6:      if M[i, n] ≤ W then                              ▷ caso base: última linha
7:          custo[i] ← 0;    prox[i] ← n + 1
8:      else
9:          custo[i] ← ∞
10:         for j ← i + 1 até n do
11:             if M[i, j − 1] > W then break
12:             c ← (W − M[i, j − 1])² + custo[j]
13:             if c < custo[i] then
14:                 custo[i] ← c;    prox[i] ← j
15: sol ← ∅;    i ← 1                                    ▷ reconstrução da solução
16: while prox[i] ≤ n do
17:     i ← prox[i]
18:     sol ← sol ∪ {i}
19: return (sol, custo[1])
```

**Exemplo.** Para a entrada do enunciado ($W = 12$), o algoritmo obtém $custo[1] = 33$ com quebras em $\{3, 5, 8, 11\}$. Logo, a solução apresentada no enunciado é ótima.

# (e) Complexidade de tempo

- **Preenchimento de $M$:** $\sum_{i=1}^{n}(n - i + 1) = O(n^2)$.
- **Preenchimento de *custo* e *prox*:** são $n$ subproblemas (item (a)); cada um examina no máximo $n - i$ candidatos $j$, cada um em $O(1)$ (a linha 10 do recursivo vira a atribuição $prox[i] \leftarrow j$, de custo constante). Total: $\sum_{i=1}^{n} O(n - i) = O(n^2)$.
- **Reconstrução:** cada passo avança $i$ estritamente, logo $O(n)$.

Portanto, o tempo total é

$$T(n) = O(n^2) + O(n^2) + O(n) = O(n^2),$$

polinomial, em contraste com o tempo exponencial do backtracking. O espaço é $O(n^2)$ por causa de $M$ (os vetores *custo* e *prox* ocupam $O(n)$).